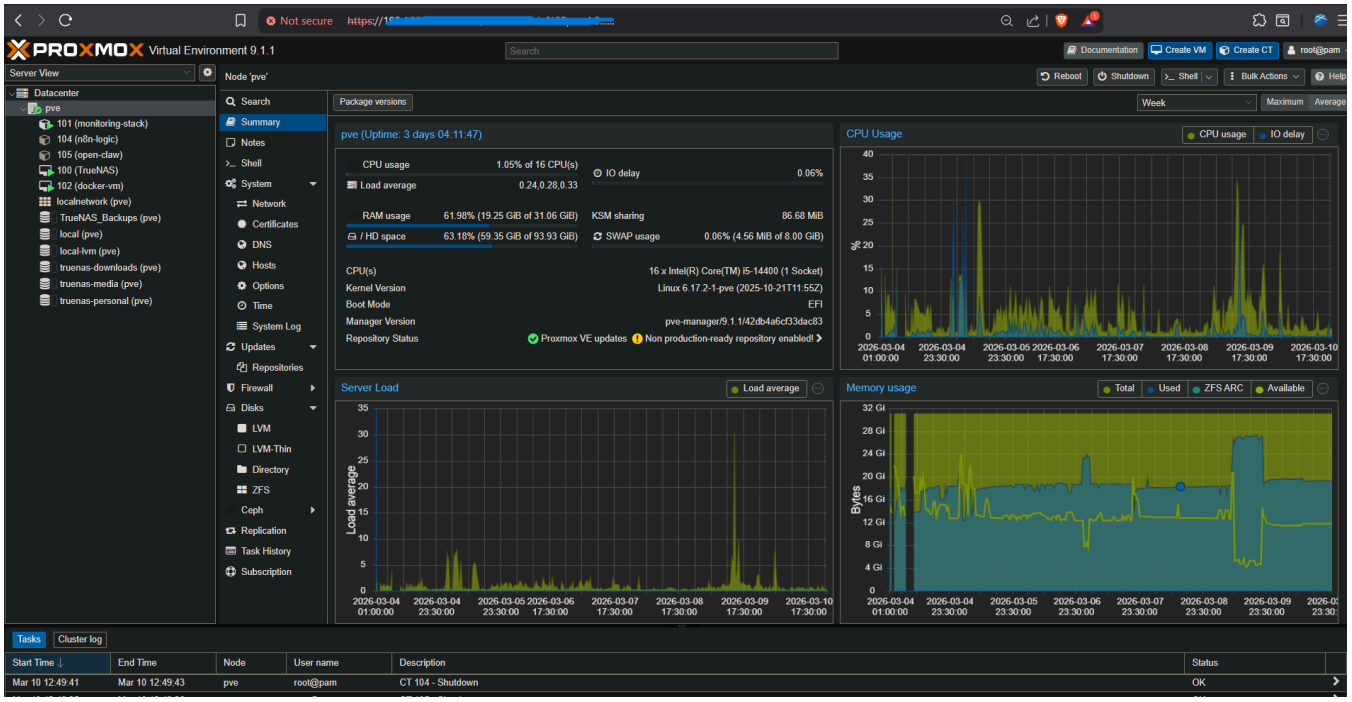
Infrastructure Foundation: Bare-Metal Hardware & Proxmox VE

Project Overview

The foundation of any sovereign cloud environment is the bare-metal hardware and the hypervisor that manages it. For this build, I required a system capable of 24/7 operation, providing enough compute for a dozen containerized services, and offering direct hardware access for a dedicated storage array. I selected **Proxmox Virtual Environment (VE)** to abstract the physical hardware and provide high-availability infrastructure for my lab.



The Hardware Stack

To balance power efficiency with enterprise-grade storage capabilities, the physical server was built with the following specifications:

- **Compute:** `Intel i5 14400`
- **Memory:** 32GB RAM
- **Primary Drive (OS/VMs):** `2TB NVMe Gen4 SSD]`
- **Mass Storage Array:** 3x 4TB Seagate IronWolf NAS HDDs
- **Networking:** 2.5 GbE NIC

Installation & Architectural Design

The goal of this layer was complete hardware abstraction. Rather than installing services directly onto a single operating system, Proxmox allows me to compartmentalize my stack.

1. **Bare-Metal OS:** Installed Proxmox VE 8.x directly to the NVMe SSD.
2. **Network Bridging:** Configured the default `vmbr0` Linux bridge to allow virtual machines to pull IPs

directly from my physical router network.

3. **Storage Tiering:** Designed a strict separation of concerns. The NVMe SSD handles high-IOPS workloads (VM disks, container root file systems, and databases), while the mechanical IronWolf HDDs are passed through strictly for bulk storage and media.

Engineering Challenges & Solutions

Challenge: The IOwait Bottleneck and System Stalls

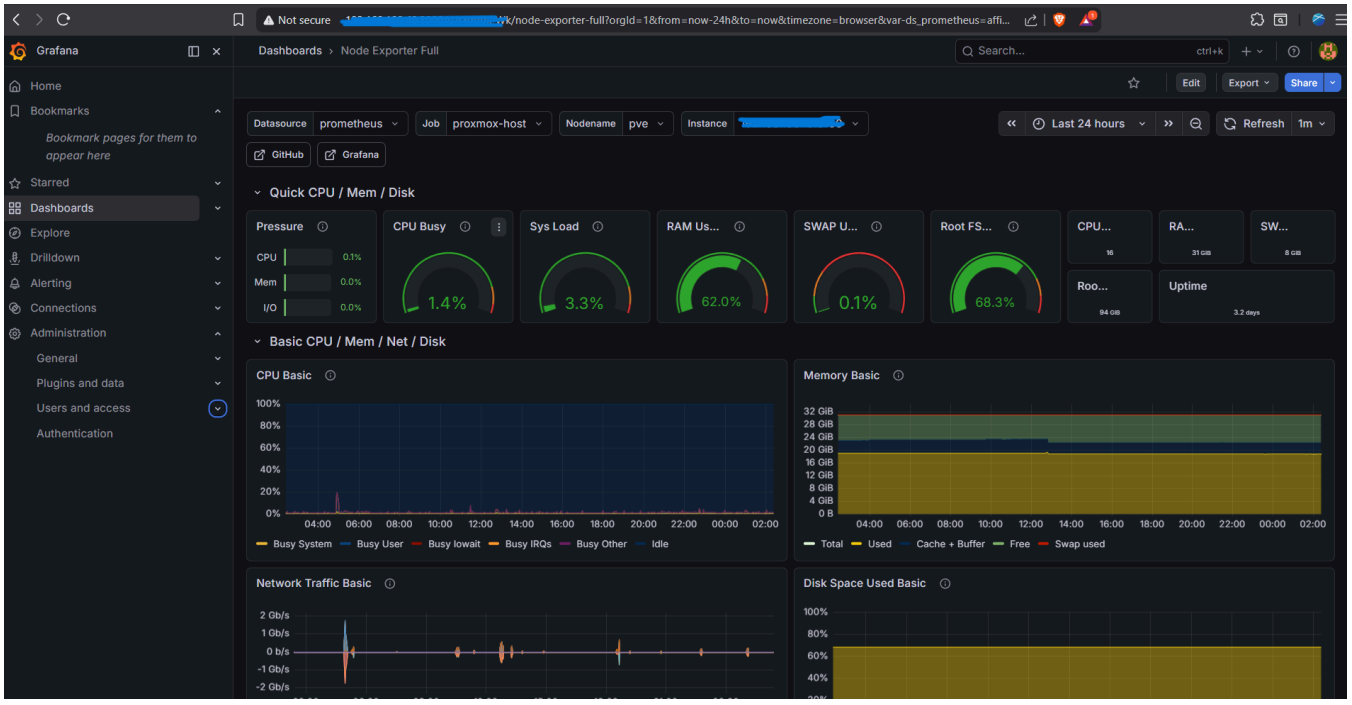
During the initial build phase, I attempted to synchronize a massive dataset (several terabytes of media and backups) directly to the spinning HDD array while my Docker VMs were also running off the same mechanical disks.

This resulted in a critical spike in **IOwait** (Input/Output Wait). The CPU was essentially idling, waiting for the mechanical read/write heads of the HDDs to catch up. Because the hypervisor, the VMs, and the data sync were all fighting for the same limited IOPS (Input/Output Operations Per Second), the entire Proxmox host experienced massive latency, causing web interfaces to timeout and containers to crash.

The Fix: Implementing an SSD Middle Layer

To resolve the IOwait lockups, I had to redesign the data flow. Mechanical drives are excellent for sequential writes but terrible at random I/O.

1. **Relocating Compute:** I migrated all Proxmox VM virtual disks (`.qcow2`/RAW files) exclusively to the NVMe SSD.
2. **The Staging Cache:** For large file synchronization, I utilized the SSD as a "middle layer" or staging ground. Data is initially ingested and processed on the high-speed NVMe drive.
3. **Scheduled Offloading:** Once processed, the data is written sequentially to the TrueNAS ZFS array on the IronWolf drives during off-peak hours, or handled natively by ZFS transaction groups in memory before hitting the disks.



What I Learned

As a System Support Engineer, I frequently encounter "slow" servers in enterprise environments. Experiencing and resolving an IOwait lockup firsthand completely changed my understanding of system bottlenecks.

I learned that throwing more CPU cores or RAM at a server will not fix a physical disk limitation. By separating my compute layer (SSD) from my storage layer (HDD), I bridged the gap between basic server setup and actual systems architecture. This foundational stability is what allowed me to confidently build out the TrueNAS and cybersecurity monitoring layers on top of it.